نظام إدارة الموظفين باستخدام PHP

1. هيكل الفئات باستخدام الوراثة

php

```php
<?php
// Employee.php - الفئة الأساسية
abstract class Employee {
    protected $id;
    protected $name;
    protected $department;
    protected $baseSalary;
    protected $bankAccount;

    public function __construct($id, $name, $department, $baseSalary) {
        $this->id = $id;
        $this->name = $name;
        $this->department = $department;
        $this->baseSalary = $baseSalary;
    }

    abstract public function calculateNetSalary();
    abstract public function getEmployeeType();
}

// PermanentEmployee.php - موظف دائم
class PermanentEmployee extends Employee {
    private $annualLeaveDays;
    private $pensionContribution;
    private $insurance = 0.14; // 14% تأمينات

    public function __construct($id, $name, $department, $baseSalary, $annualL
eaveDays) {
        parent::__construct($id, $name, $department, $baseSalary);
        $this->annualLeaveDays = $annualLeaveDays;
    }

    public function calculateNetSalary() {
```

```php
        $deductions = $this->baseSalary * $this->insurance;
        return $this->baseSalary - $deductions;
    }

    public function getEmployeeType() {
        return "موظف دائم";
    }

    // Getter و Setter محمي
    public function getPensionContribution() {
        return $this->pensionContribution;
    }

    public function setPensionContribution($value) {
        $this->pensionContribution = $value;
    }
}

// ContractEmployee.php - موظف بعقد
class ContractEmployee extends Employee {
    private $contractEndDate;
    private $contractType;
    private $taxRate = 0.05; // 5% ضرائب فقط

    public function __construct($id, $name, $department, $baseSalary, $contrac
tEndDate) {
        parent::__construct($id, $name, $department, $baseSalary);
        $this->contractEndDate = $contractEndDate;
    }

    public function calculateNetSalary() {
        $deductions = $this->baseSalary * $this->taxRate;
        return $this->baseSalary - $deductions;
    }

    public function getEmployeeType() {
        return "موظف بعقد";
    }
```

```php
    public function isContractActive() {
        return strtotime($this->contractEndDate) > time();
    }
}

// Manager.php - المدير (يرث من الموظف الدائم)
class Manager extends PermanentEmployee {
    private $managementAllowance;
    private $teamSize;
    private $bonuses = [];

    public function __construct($id, $name, $department, $baseSalary, $annualLeaveDays, $teamSize) {
        parent::__construct($id, $name, $department, $baseSalary, $annualLeaveDays);
        $this->teamSize = $teamSize;
        $this->managementAllowance = $baseSalary * 0.20; // 20% بدل إدارة
    }

    public function calculateNetSalary() {
        $baseNet = parent::calculateNetSalary();
        $bonusTotal = array_sum($this->bonuses);
        return $baseNet + $this->managementAllowance + $bonusTotal;
    }

    public function addBonus($amount, $reason) {
        $this->bonuses[] = [
            'amount' => $amount,
            'reason' => $reason,
            'date' => date('Y-m-d')
        ];
    }

    public function getEmployeeType() {
        return "مدير";
    }
}
?>
```

2. حماية بيانات الرواتب بالتغليف

php

```php
<?php
// SalaryProtector.php
class SalaryProtector {
    private $employeesSalaries = [];
    private $accessLog = [];
    private $allowedRoles = ['HR', 'Finance', 'Manager'];

    public function setEmployeeSalary($employeeId, $salary, $approverRole, $approverId) {
        if (!in_array($approverRole, ['HR_Manager', 'CEO'])) {
            throw new Exception("لا تملك صلاحية تعديل الرواتب");
        }

        $this->employeesSalaries[$employeeId] = [
            'salary' => $salary,
            'last_updated' => date('Y-m-d H:i:s'),
            'updated_by' => $approverId
        ];

        $this->logAccess($employeeId, "تعديل راتب", $approverId, $approverRole);
    }

    public function getEmployeeSalary($employeeId, $requesterRole, $requesterId) {
        // التحقق من الصلاحيات
        if (!in_array($requesterRole, $this->allowedRoles) && $requesterId != $employeeId) {
            throw new Exception("غير مصرح بالوصول لهذه البيانات");
        }

        if (!isset($this->employeesSalaries[$employeeId])) {
            throw new Exception("الراتب غير مسجل");
        }
```

```
        $this->logAccess($employeeId, "استعلام عن راتب", $requesterId, $reques
terRole);

        return [
            'salary' => $this->employeesSalaries[$employeeId]['salary'],
            'currency' => 'SAR'
        ];
    }

    private function logAccess($employeeId, $action, $userId, $userRole) {
        $this->accessLog[] = [
            'timestamp' => date('Y-m-d H:i:s'),
            'employee_id' => $employeeId,
            'action' => $action,
            'user_id' => $userId,
            'user_role' => $userRole,
            'ip_address' => $_SERVER['REMOTE_ADDR'] ?? 'N/A'
        ];
    }

    public function getAccessLog($adminRole) {
        if ($adminRole !== 'System_Admin') {
            throw new Exception("صلاحية إدارية مطلوبة");
        }
        return $this->accessLog;
    }
}
?>
```

3. تعدد الأشكال في حساب الرواتب

php

```
<?php
// PayrollCalculator.php
class PayrollCalculator {
    private $employees = [];

    public function addEmployee(Employee $employee) {
```

```
        $this->employees[] = $employee;
    }

    public function calculateTotalPayroll() {
        $total = 0;
        $details = [];

        foreach ($this->employees as $employee) {
            $netSalary = $employee->calculateNetSalary();
            $total += $netSalary;

            $details[] = [
                'id' => $employee->id,
                'name' => $employee->name,
                'type' => $employee->getEmployeeType(),
                'net_salary' => $netSalary
            ];
        }

        return [
            'total_payroll' => $total,
            'employee_count' => count($this->employees),
            'details' => $details
        ];
    }

    public function generateSalarySlip(Employee $employee) {
        $netSalary = $employee->calculateNetSalary();

        return [
            'employee_id' => $employee->id,
            'employee_name' => $employee->name,
            'employee_type' => $employee->getEmployeeType(),
            'month' => date('F Y'),
            'net_salary' => $netSalary,
            'payment_date' => date('Y-m-27'),
            'bank_account' => substr($employee->bankAccount, -4) // آخر 4 أرقام فقط
        ];
```

```
    }
}
?>
```

4. الواجهات لضمان سلوك موحد

php

```php
<?php
// Interfaces.php

interface Workable {
    public function assignTask($task);
    public function completeTask($taskId);
    public function getCompletedTasks();
}

interface Evaluatable {
    public function setPerformanceGoals($goals);
    public function calculatePerformanceScore();
    public function getEvaluationReport();
}

interface Reportable {
    public function generateMonthlyReport();
    public function submitReport($report);
}

// تطبيق الواجهات
class Manager extends PermanentEmployee implements Workable, Evaluatable, Repo
rtable {
    private $tasks = [];
    private $performanceGoals = [];
    private $reports = [];

    // تنفيذ واجهة Workable
    public function assignTask($task) {
        $this->tasks[] = [
            'task' => $task,
```

```
            'assigned_at' => date('Y-m-d H:i:s'),
            'completed' => false
        ];
    }

    public function completeTask($taskId) {
        if (isset($this->tasks[$taskId])) {
            $this->tasks[$taskId]['completed'] = true;
            $this->tasks[$taskId]['completed_at'] = date('Y-m-d H:i:s');
        }
    }

    public function getCompletedTasks() {
        return array_filter($this->tasks, function($task) {
            return $task['completed'];
        });
    }

    // تنفيذ واجهة Evaluatable
    public function setPerformanceGoals($goals) {
        $this->performanceGoals = $goals;
    }

    public function calculatePerformanceScore() {
        $completedTasks = count($this->getCompletedTasks());
        $totalTasks = count($this->tasks);

        if ($totalTasks === 0) return 0;

        return ($completedTasks / $totalTasks) * 100;
    }

    public function getEvaluationReport() {
        return [
            'performance_score' => $this->calculatePerformanceScore(),
            'goals' => $this->performanceGoals,
            'tasks_completed' => count($this->getCompletedTasks()),
            'total_tasks' => count($this->tasks)
        ];
```

```
    }

    // تنفيذ واجهة Reportable
    public function generateMonthlyReport() {
        return [
            'department' => $this->department,
            'month' => date('F Y'),
            'team_performance' => $this->calculatePerformanceScore(),
            'team_size' => $this->teamSize,
            'achievements' => $this->getCompletedTasks()
        ];
    }

    public function submitReport($report) {
        $this->reports[] = [
            'report' => $report,
            'submitted_at' => date('Y-m-d H:i:s')
        ];
    }
}
?>
```

5. المزايا الإضافية للمديرين (Composition)

php

```
<?php
// ManagerBenefits.php - استخدام التركيب بدلاً من الوراثة المفرطة
class ManagerBenefits {
    private $carAllowance = 2000;
    private $housingAllowance = 3000;
    private $educationAllowance = 1000;
    private $healthInsurance = 'Premium';
    private $clubMemberships = [];

    public function calculateTotalBenefits() {
        return $this->carAllowance + $this->housingAllowance + $this->educatio
nAllowance;
    }
```

```php
    public function addClubMembership($clubName, $annualFee) {
        $this->clubMemberships[] = [
            'club' => $clubName,
            'fee' => $annualFee,
            'added_date' => date('Y-m-d')
        ];
    }

    public function getBenefitsPackage() {
        return [
            'car_allowance' => $this->carAllowance,
            'housing_allowance' => $this->housingAllowance,
            'education_allowance' => $this->educationAllowance,
            'health_insurance' => $this->healthInsurance,
            'club_memberships' => $this->clubMemberships,
            'total_monthly_benefits' => $this->calculateTotalBenefits()
        ];
    }
}

// Manager class with benefits composition
class ManagerWithBenefits extends PermanentEmployee {
    private $benefits;
    private $teamSize;

    public function __construct($id, $name, $department, $baseSalary, $annualL
eaveDays, $teamSize) {
        parent::__construct($id, $name, $department, $baseSalary, $annualLeave
Days);
        $this->teamSize = $teamSize;
        $this->benefits = new ManagerBenefits();
    }

    public function calculateTotalCompensation() {
        $baseSalary = $this->calculateNetSalary();
        $benefits = $this->benefits->calculateTotalBenefits();
        return $baseSalary + $benefits;
    }
```

```php
    public function getCompensationBreakdown() {
        return [
            'base_salary' => $this->baseSalary,
            'net_salary' => $this->calculateNetSalary(),
            'benefits' => $this->benefits->getBenefitsPackage(),
            'total_compensation' => $this->calculateTotalCompensation()
        ];
    }

    // يمكن إضافة المزايا بسهولة
    public function addClubMembership($clubName, $annualFee) {
        $this->benefits->addClubMembership($clubName, $annualFee);
    }
}
?>
```

6. التطبيق الرئيسي والنماذج

php

```php
<?php
// index.php - التطبيق الرئيسي
require_once 'Employee.php';
require_once 'PermanentEmployee.php';
require_once 'ContractEmployee.php';
require_once 'ManagerWithBenefits.php';
require_once 'PayrollCalculator.php';
require_once 'SalaryProtector.php';

// إنشاء موظفين
$emp1 = new PermanentEmployee(101, "أحمد محمد", "المبيعات", 8000, 21);
$emp2 = new ContractEmployee(102, "سارة علي", "التسويق", 6000, "2024-12-31");
$manager = new ManagerWithBenefits(201, "خالد عبدالله", "الإدارة", 15000, 30, 5);

// إضافة مزايا للمدير
$manager->addClubMembership("نادي الأعمال", 5000);
$manager->addBonus(2000, "تحقيق أهداف الربع");
```

```php
// حاسبة الرواتب
$payroll = new PayrollCalculator();
$payroll->addEmployee($emp1);
$payroll->addEmployee($emp2);
$payroll->addEmployee($manager);

// حساب إجمالي الرواتب
$totalPayroll = $payroll->calculateTotalPayroll();

// حماية بيانات الرواتب
$salaryProtector = new SalaryProtector();
try {
    $salaryProtector->setEmployeeSalary(101, 8000, "HR_Manager", 1);
    $emp1Salary = $salaryProtector->getEmployeeSalary(101, "Manager", 201);
} catch (Exception $e) {
    echo "خطأ: " . $e->getMessage();
}

// عرض النتائج
echo "<h2>إدارة الموظفين</h2>";
echo "<h3>تفاصيل الموظفين:</h3>";

$employees = [$emp1, $emp2, $manager];
foreach ($employees as $emp) {
    echo "<div style='border:1px solid #ccc; padding:10px; margin:10px;'>";
    echo "<strong>اسم الموظف:</strong> " . $emp->name . "<br>";
    echo "<strong>النوع:</strong> " . $emp->getEmployeeType() . "<br>";
    echo "<strong>القسم:</strong> " . $emp->department . "<br>";
    echo "<strong>صافي الراتب:</strong> " . $emp->calculateNetSalary() . " ﷼<
br>";

    if ($emp instanceof ManagerWithBenefits) {
        $compensation = $emp->getCompensationBreakdown();
        echo "<strong>إجمالي التعويضات:</strong> " . $compensation['total_com
pensation'] . " ﷼<br>";
    }
    echo "</div>";
}
```

```
echo "<h3>إجمالي كشوف المرتبات:</h3>";
echo "<strong>إجمالي الرواتب:</strong> " . $totalPayroll['total_payroll'] . "
ريال<br>";
echo "<strong>عدد الموظفين:</strong> " . $totalPayroll['employee_count'] . "<
br>";

echo "<h3>كشف رواتب مفصل:</h3>";
foreach ($totalPayroll['details'] as $detail) {
    echo $detail['name'] . " (" . $detail['type'] . "): " . $detail['net_salar
y'] . " ريال<br>";
}
?>
```

7. قاعدة البيانات (MySQL)

```sql
-- employees.sql
CREATE TABLE employees (
    id INT PRIMARY KEY AUTO_INCREMENT,
    employee_type ENUM('permanent', 'contract', 'manager'),
    name VARCHAR(100) NOT NULL,
    department VARCHAR(50),
    base_salary DECIMAL(10,2),
    annual_leave_days INT,
    contract_end_date DATE,
    team_size INT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE salaries (
    id INT PRIMARY KEY AUTO_INCREMENT,
    employee_id INT,
    month YEAR_MONTH,
    net_salary DECIMAL(10,2),
    deductions DECIMAL(10,2),
    allowances DECIMAL(10,2),
    status ENUM('pending', 'paid', 'cancelled'),
    paid_at DATETIME,
```

```
    FOREIGN KEY (employee_id) REFERENCES employees(id)
);

CREATE TABLE manager_benefits (
    id INT PRIMARY KEY AUTO_INCREMENT,
    manager_id INT,
    benefit_type VARCHAR(50),
    amount DECIMAL(10,2),
    start_date DATE,
    end_date DATE,
    FOREIGN KEY (manager_id) REFERENCES employees(id)
);
```

مزايا هذا التصميم:

1. **الوراثة**: إعادة استخدام الكود في الفئات الأساسية
2. **التغليف**: حماية بيانات الرواتب الحساسة
3. **تعدد الأشكال**: حساب الرواتب بطريقة مختلفة لكل نوع
4. **الواجهات**: ضمان سلوك موحد للمديرين
5. **التركيب (Composition)**: إضافة مزايا دون كسر مبدأ الوراثة الواحدة
6. **المرونة**: إضافة أنواع جديدة بسهولة